```
  val name = "ContainerService"
...
  val addCargoDescriptor: MethodDescriptor[
      example.akka.grpc.Cargo, example.akka.grpc.AddedCargo] =
        MethodDescriptor.newBuilder()
        .setType(MethodDescriptor.MethodType.UNARY)                    ❶
        .setFullMethodName(
          MethodDescriptor
          .generateFullMethodName("ContainerService","AddCargo"))
        .setRequestMarshaller(new Marshaller(CargoSerializer))         ❷
        .setResponseMarshaller(new Marshaller(AddedCargoSerializer     ❸
        ...
```

❶ Unary call: one request, one response

❷ Marshalling/unmarshalling request

❸ Marshalling/unmarshalling response

Thanks to the autogenerated marshallers, you have direct access to Scala objects like cargo in `addCargo(in: Cargo)`.

11.2.4 Akka gRPC in action

You need to implement the `addCargo` method from the previous trait. To implement this dummy service, you can return a completed `Future[AddedCargo]`. To do so, implement the `ContainerService`; and for the execution of the future, you need an implicit `executionContext` that you can add in the constructor of the implementation of the service. To return the completed future, use `Future.successful([response])`:

```
import example.container.grpc.{ AddedCargo, Cargo, ContainerService }
import scala.concurrent.{Future, ExecutionContext}

class ContainerServiceImpl(implicit executionContex: ExecutionContext)   ❶
    extends ContainerService {
```